

Министерство образования и науки Российской Федерации
Московский физико-технический институт (национальный
исследовательский университет)

Факультет радиотехники и компьютерных технологий
Кафедра

Выпускная квалификационная работа бакалавра по направлению 010900
«Прикладные математика и физика»

Изучение мохнатых существ с α Центавра

Студент 082 группы
Сидоров А. Б.

Научный руководитель
Иванов В. Г.

Долгопрудный
2026

Содержание

1. Введение	1
2. Постановка проблемы	3
2.1. Цели работы:	3
2.2. Задачи работы:	3
3. Обзор существующих решений	5
3.1. Верификация байт-кода статически типизированных яп:	5
3.2. Верификация байт-кода динамически типизированных яп:	7
Благодарности	9

Глава 1

Введение

В настоящее время создается множество языков программирования, каждый из которых своей целью ставит удобство решения определенного класса задач. Большинство ныне существующих языков программирования относятся к платформо-независимым языкам. Главным их преимуществом является высокая переносимость кода, существует идиома описывающая данный вид поведения (WORA — Write Once, Run Anywhere). Примерами таких языков являются: ArkTS, C# Dart, Java.

Высокая переносимость обеспечивается по средствам специальных сред исполнения, называемых виртуальными машинами, которые исполняют байт-код. Байт-код — промежуточный, платформенно-независимый код низкого уровня, получаемый при компиляции исходного текста программы. Он состоит из компактных инструкций, которые исполняются не напрямую процессором, а виртуальной машиной (например, JVM для Java), что позволяет запускать приложения на Windows, Linux, macOS и других платформах без перекомпиляции.

В связи со сказанным выше на первый план выходит проблема отслеживания корректности (здесь и далее под корректностью будем понимать соответствие архитектуре набора команд и семантике языка), сгенерированного байт-кода. Программы отслеживающие корректность называются верификаторами байт-кода. Важность изучения и разработки данных программы обуславливается тем, что исполняемые приложения могут отвечать за функционирование крайне чувствительных областей, таких как телекоммуникации, банковская сфера, медицинские услуги, где ошибка исполнения может привести к необратимому ущербу. Наличие верификатора позволяет обнаруживать ошибки до времени исполнения. Кроме того, верификатор оказывается полезен на этапе разработки языка программирования.

Существует несколько источников некорректного байт-кода:

1. Ошибка компилятора на этапе кодогенерации, Lowering (понижение), регистровой аллокации и т.д.
2. Повреждения пакета с исполняемым кодом во время его передачи по сети.

В данной работе фокус идет на повышение точности работы верификатора байт-кода путем расширения его типовой система и увеличения множества верифицируемых инструкций.

Глава 2

Постановка проблемы

2.1 Цели работы:

1. Расширить типовую систему верификатора для более точного отображения типовой системы виртуальной машины на типовую систему верификатора.
2. Разработать и внедрить алгоритмы проверок ряда инструкций, для расширения множества верифицируемых программ.

2.2 Задачи работы:

1. Изучить существующие решения.
2. Сравнить типовые системы виртуальной машины и верификатора, выявить неверифицируемые типы.
3. Расширить типовую системы и поддержать логику обработки данных типов.
4. Проанализировать архитектуру набора команд платформы и выявить множество неверифицируемых инструкций.
5. Реализовать и внедрить алгоритм верификации новых инструкций.
6. Протестировать каждый из предложенных алгоритмов.

Цели работы можно считать достигнутыми. Так как по ходу работы было реализовано порядка 10 обработок инструкций, что позволило выявить ошибки кодогенерации компилятора и улучшить гарантии платформы по безопасности. Как итог ahead-of-time(заранее) режим верификатора был добавлен в основную ветку OHOS

Глава 3

Обзор существующих решений

На данный момент большинство виртуальных машин обладают верификаторами, в качестве самых популярных примеров могут послужить JVM - виртуальная машина для исполнения байт-кода таких языков как: java, kotlin; .Net CLR - виртуальная машина для исполнения байт-кода C#, F#. Так же существуют верификаторы, которые не поставляются вместе с платформой, а являются сторонними библиотеками, примерами могут послужить BCEL Verifier для Java байт-кода и PEVerify, который проверяет на корректность IL-кода(C#). Рассмотрим некоторые из существующих решений и виды выполняемых проверок более подробно.

3.1 Верификация байт-кода статически типизированных ЯП:

В статически типизированных языках типы переменных, аргументов и возвращаемых значений известны на этапе компиляции. Это позволяет компилятору генерировать байткод с встроенными гарантиями типов. Но байткод может быть сгенерирован не только компилятором, поэтому верификатор остается неотъемлемой частью виртуальной машины. Перечислим основные выполняемые проверки:

1. Проверки формата байткода:

Целью данных проверок является тестирование на валидность инструкций, корректность их формата и правильность индексов, соответствующих методам, полям и типам.

2. Проверки графа управления

Во время выполнения данного типа проверок, программа анализируется на корректность инструкций условного перехода: целевой адрес не выходит за пределы текущего метода, в середину try-блока или в середину другой инструкции. Проверяется отсутствие циклов с некорректным состоянием регистров. Контро-

лируется, что все пути выполнения заканчиваются инструкциями обозначающими *return* или *throw*

3. Проверки типовой безопасности регистров:

Байткод инспектируется на предмет того, что тип каждого регистра является примитивным или ссылочным. Верифицируется, что любой регистр используется после своей инициализации.

4. Проверки инструкций вызова:

Инструкции вызова проходят проверку на корректность объекта ресивера(в случае нестатического метода). На количество аргументов, на типы данных аргументов и тип возвращаемого значения. Верифицируется возможность доступа.

5. Проверка доступа обращения:

Именно данная верификация является одной из самых важных для безопасности. Как известно, существует 3 основных модификатора доступа: публичные, защищенные и приватные, которые применимы, как к элементам класса, так и классам и целым модулям. Контроль того, что их семантика не нарушается на уровне байткода необходим.

Рассмотрим пару примеров промышленных решений

1. ART

В контексте данной работы для нас наибольший интерес представляют верификаторы тех платформ, которые предназначены для исполнения кода на мобильных устройствах. Именно таким является верификатор ART(Android runtime). ART исполняет DEX-байткод, источником которого могут быть: сторонние приложения, сгенерированного кода, потенциально повреждённых или злонамеренных источников.

2. LLVM

LLVM-верификация — это обладает несколько иной философией, чем ART: она не про безопасность пользователя, а про корректность как ПП(здесь и далее ПП - промежуточное представление) компилятора. ПП генерируется доверенным фронтендом, но оптимизации могут его сломать. Можно выделить 4 уровня верификации: уровень модуля, функций, базовых блоков и инструкций. Проверяются как локальные, так глобальные инварианты.

3.2 Верификация байт-кода динамически типизированных яп:

В динамически типизированных языках типы определяются в процессе исполнения: переменная может менять тип . Байткод здесь часто проще, и верификатор фокусируется не на статических типах, а на структурной целостности и совместимости.

Благодарности

Благодарности идут тут.